

① Explain overfitting & techniques to reduce it.

Overfitting occurs when a machine learning model learns the training data "too well," capturing the noise and random fluctuations rather than the underlying pattern.

While the model performs with near-perfect accuracy on its training set, it fails to generalize to new, unseen data. Think of it like a student who memorizes the exact answers to practice exam but doesn't understand the actual concepts; when the real test has slightly different questions, they fail.

Techniques to reduce overfitting to ensure your model stays flexible and focused on the "big picture," you can use these common strategies:

Cross-validation & Split your data into multiple subsets. Train the model on some and validate it on others to ensure performance is consistent across different data samples.

Regularization (L1 and L2): This adds a "penalty" to the loss function based on the size of the model's weights. It discourages the model from becoming overly complex by shrinking the importance of less critical features.

Data Augmentation: Increase the diversity of your training set by creating modified versions of existing data (e.g., flipping, rotating, or cropping images). This forces the model to learn the object itself rather than a specific orientation.

Dropout is specifically used in neural network, this technique randomly "turn off" a percentage of neurons during each training step. This prevents the model from relying too heavily on any single path or feature.

Early Stopping monitors the model's performance on a validation set during training. Stop the training process the moment the validation error starts to increase, even if the training error is still going down.

② Describe the bias-variance tradeoff with examples

The bias-variance tradeoff is the central struggle in machine learning to find the "sweet spot" between a model that is too simple and one that is too complex.

□ Bias (The Error of Simplification)

Bias occurs when a model makes ~~into~~ overly simplistic assumptions about the data. This leads to underfitting, where the model misses the relevant relations between features and targets.

- High Bias: The model is "stubborn." even with more data, it fails to capture the complexity

- Example: Trying to predict house prices using only the number of rooms, while ignoring location, age, and square footage. The model will be consistently wrong because it's too basic.

□ Variance (The Error of Sensitivity)

Variance occurs when a model is overly sensitive to the specific noise and fluctuations in the training

st. This leads to overfitting, where the model performs perfectly on training data but fails on new data.

- o High variance: The model is "disgorged." it treats random noise as if it were a crucial pattern.
- o Example: A model that memorizes the exact lighting and background of every leaf photo in an agri-tech dataset. If new photo has a different background, the model gets confused because it learned the "noise" of the old photos.

3] The Trade off

The goal is to minimize both errors. As you decrease bias (by making the model more complex), variance typically increases. Conversely, as you decrease variance (by simplifying the model), bias increases.

Feature	Underfitting	Overfitting
Complexity	Too low	Too high
Training error	High	Very low
Test error	High	High
Solution	add features, use more complex model.	use more data, regularization, or simpler model.

3] Why is feature scaling important? which algorithm require it?

Feature scaling is important because it standardizes the range of independent variables, preventing features with larger values from dominating the learning process.

Algorithm Algorithms that calculate distance

between data points or depend on the magnitude of features require it most. This includes:

- K-nearest neighbors (KNN).
- Support vector machines (SVMs).
- K-Means clustering.
- Principle Component Analysis (PCA).
- Gradient Descent-based algorithms (like Linear Regression, logistic Regression, neural networks).

(4) Compare classification and regression with suitable use cases?

The difference between classification and regression comes down to the type of "answers" you are looking for: a category or a number.

[1] Classification (predicting "which one?")

Classification is used when the output variable is ~~discrete~~ discrete or categorical. The goal is to map input data into specific predefined labels or classes:

- Output $\Rightarrow$ labels, categories, or "buckets".
- Key question $\Rightarrow$ Does this belong to A or B.
- Algorithms $\Rightarrow$ logistic Regression, SVMs, KNN, etc.

Use cases $\Rightarrow$

- Email filtering
- Medical Diagnostics
- Image Recognition

[2] Regression (predicting "How much?")

Regression is used when the output variable is continuous or numerical. The goal is to find a mathematical relationship between inputs and a specific value on a scale.

output type = Real numbers (integers or decimals).
 Key question = what is the specific value of the output based on these variables?

Algorithms : linear Regression, polynomial Regression
 Ridge / Lasso Regression.

Use cases :

- Real state
- Weather forecasting
- Finance

⑤ What is cross-validation and how does it improves model reliability?

Cross-validation (CV) is a resampling technique used to evaluate how well a machine learning model generalizes to an independent dataset. Instead of splitting your data just once into a "training" and "testing" set, cross-validation involves partitioning the data multiple times and rotating which part is used for training versus testing.

The most common version is K-fold cross-validation, where the data is split into K equal segments (folds). The model is trained on K-1 folds and tested on the remaining fold. This process repeats K times until every data point has been used for testing exactly once.

How it Improves Model Reliability?

- Reduces "luck of the draw"
- Detects overfitting.
- Optimizes Hyperparameters.
- Provides Confidence Intervals.

⑥ A model performs well on training data

but poorer on test data. Analyze the issue and suggest improvements.

This scenario is a classic case of overfitting. Essentially your model has "memorized" the noise and specific patterns of the training set rather than learning the underlying general principles that apply to new data. The problem is high variance & when a model overfits it becomes overly complex relative to the amount and noise of the data. It treats random fluctuations as significant features.

- Suggest of Improvements : to fix overfitting we need to simplify the model or provide it with more context to generalize better :
 - Regularization (L_1 or L_2)
 - Simplify the Architecture
 - Increase Training Data.
 - Early Stopping
 - Dropout
 - Cross-validation

⑦ You given a dataset with highly correlated features. How would you handle this during model building?

Handling highly correlated features - a situation known as multicollinearity - is crucial because it can inflate the variance of ~~coefficient~~ coefficient estimates, making your model unstable and difficult to interpret.

III Detect the Redundancy

- Correlation matrix & use heatmap to visually identify pairs (typically $|x| > 0.8$ or 0.9)

- Variance Inflation factor (VIF) value exceeding 5 or 10 generally indicates problematic correlation.

2 Feature Selection (Removal)

- Manual Drop & Drop missing values.
- Automated Selection & Use techniques like RFE to let model decide which features drop.

3 Dimensionality Reduction

Instead of Throwing data away, you can transform

- principle Component Analysis (PCA).
- Feature Engineering

4 Use Regularization

If you are using linear models, specific "penalties" can handle correlation automatically:

- Ridge Regression (L_2).
- Lasso Regression (L_1).
- Elastic net (hybrid of both L_1 and L_2)

5 Switch model type

Some ~~types~~ algorithms are naturally robust to multicollinearity (e.g. like tree-based models). Random Forest and Gradient Boosting machine (XGBoost, LightGBM) handle correlated features better than linear models because they choose one feature to split on at a time, though correlation can still make "feature importance" scores somewhat misleading.

Chapter 2 ML Evaluation Metrics

① Define precision and recall with use cases.

In classification tasks, precision and recall are metrics used to evaluate a model's performance beyond simple accuracy, especially when dealing with imbalanced datasets.

1] Precision (Quality)

Precision measures the accuracy of the positive predictions. It answers "of all instances the model predicted as positive, how many were actually positive?"

$$\text{Precision} = \frac{\text{True Positives (TP)}}{\text{True Positives (TP) + False Positives (FP)}}$$

- Goal: minimize false positive.
- Use case: Email spam detection.

2] Recall (Quantity)

Recall (also called Sensitivity) measures the model's ability to find all positive instances. It answers "of all the actual positive instances that exists, how many did the model identify?"

$$\text{Recall} = \frac{\text{True Positives (TP)}}{\text{True Positives (TP) + False Negatives (FN)}}$$

- Goal: minimize false negatives.
- Use case: Cancer diagnosis.

3] Explain F1-Score and why it is preferred for imbalanced data sets.

The F1-Score is a single metric that combines two competing aspects of model's performance: precision and Recall. It is specifically designed to provide a balanced view when you care about both finding all positives instances and ensuring those you find are actually correct.

$$F_1 = 2 \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

Using the harmonic mean instead of a simple average ensures that if either precision or Recall is very low (near zero), the F1-Score will also be low.

Why is preferred?

In an imbalanced data set (e.g., a fraud detection set where 99% of transactions are legitimate), standard Accuracy becomes a "liar".

- The Accuracy Trap: If you build a model that simply predicts "not fraud" for every single case, you would achieve 99% accuracy. However, the model is completely useless because it failed to catch any single fraudulent transaction.
- The F1-Score Solution: In the same scenario, the model's Recall would be 0, which forces the F1-score to be 0.

Key Advantages

III penalizes extremes: It prevents you from favoring a model that is "all talk" (high precision, low recall) or "guesswork" (high recall, low precision).

[2] Focuses on the minority class: In most imbalanced problems, the "positive" class is the rare one we really care about. F1-score centres the evaluation on how well the model handles that specific class.

③ What is a confusion matrix and how is it interpreted?

A confusion matrix is a performance measurement tool used in ML to evaluate the accuracy of a classification model. It is a square table that compares the actual target values with those predicted by the model.

The Four Quadrants

In a binary classification (e.g., "Sick" vs "Healthy") the matrix is divided into four outcomes:

- True Positive (TP): You predicted positive and it's actually positive.
- True Negative (TN): You predicted negative and it's actually negative.
- False Positive (FP): Type I error. You predicted positive, but it's actually negative ~~error~~.
- False Negative (FN): Type II error. You predicted negative, but it's actually positive.

How to Interpret It?

To interpret the matrix, you look at where the model is "confused." A perfect model would have values only on the main diagonal (top-left to bottom-right), meaning all predictions match the actual results.

- Accuracy = $(TP + TN) / \text{total} \Rightarrow$ over all, how often is the classifier correct?
- Precision = $(TP / TP + FP) \Rightarrow$ when it predicts positive, how often it's right?
- Recall = $(TP / TP + FN) \Rightarrow$ out of all actual positives, how many did it find?
- F1-Score = $\frac{2(\text{Precision} \times \text{Recall})}{(\text{Precision} + \text{Recall})} \Rightarrow$ The

harmonic mean of precision and recall.

Why not just use Accuracy?

Accuracy can be misleading if your classes are imbalanced. For example, if 99% of people are healthy, a model that predicts "healthy" for everyone will be 99% accurate but completely useless at finding sick people. The confusion matrix exposes this by showing a high number of false negatives.

④ Explain ROC curve and AUC.

The ROC Curve (Receiver Operating Characteristic) and AUC (Area Under the Curve) are tools used to evaluate a classification model's performance at various threshold settings.

1 ROC : A ROC is graph that plots the trade off between the "hits" and "false alarms" of the model.

True Positive Rate (Recall) = $TP / (TP + FN)$

• The ability to find positives.

False Positive Rate = $FP / (FP + TN)$

• The rate at which negatives are wrongly flagged as positives.

The curve plots these values at every possible classification threshold (from 0 to 1).

2 AUC : It provides an aggregate measure of performance across all possible classification

• Interpretation : Think of AUC as probability that the model will rank a randomly chosen positive instance higher than a randomly chosen negative one.

• The Scale : 1.0 A perfect model (The curve reaches the top-left corner).

0.5 : A worthless model (Guessing).

0.0 : A perfectly "inverted" model (it gets everything wrong)

5 When should accuracy not be used as metric?
Accuracy is a deceptive metric whenever you are dealing with imbalanced datasets, situation where one class significantly outnumbers the other.

In this cases, a model can achieve "high accuracy" simply by predicting the majority class every time.

Also, accuracy should be used when the cost of false

Negative is much higher than false positive or vice versa. Accuracy accounts all errors equally, but in the real-world:

- missing a cancer diagnosis (FN) is much worse than false alarm (FP).

- In case these cases, you should prioritize Recall or precision instead.

⑥ In a medical diagnosis system, false negative are critical, which metric would you prioritize and why?

In a medical diagnosis system, you must prioritize Recall (also known as sensitivity). Recall measures the model's ability to identify all actual positive cases and maximizing it directly minimizes false negatives. In medicine, a false negative means a sick patient is sent home undiagnosed, which can lead to untreated illness or death. While high Recall may increase false positives (false alarms), these generally suffer as they can be cleared by further testing. Therefore, ensuring no true cases are missed is the highest priority.

⑦ A model has high accuracy but poor recall. Analyze the situation and suggest improvements.

This model is likely getting into the "accuracy paradox" by favoring the majority class and failing to identify actual positive cases (high false negative). This often happens with imbalanced data where the model plays it safe to maintain a high correct-guess percentage.

To fix this, try lowering the classification threshold to be more sensitive or applying SMOTE to oversample the minority class. You could also use cost-sensitive learning to penalize the model more heavily for missing.

④ Chapter 3 CNN

① Explain the working of convolution layers?

Convolutional layers use filters (kernels) that slide across an input image to perform element-wise multiplication and sum summation. This process, known as a convolution operation, produces feature maps that highlight specific patterns like edges, textures, or shapes. By sharing weights across the entire image, these layers significantly reduce the number of parameters compared to fully connected layers. They maintain spatial hierarchy, meaning they understand how pixels relate to their neighbors rather than treating them as isolated data points. As the network gets deeper, these layers start to recognize increasingly complex features, moving from simple lines to entire objects.

② What are the purpose of pooling layers?

The primary purpose of pooling layers is to reduce the spatial dimensions (width and height) of the input data, effectively downsampling the feature maps. By doing so, it decreases the number of parameters and computational load in the network, which helps prevent overfitting. It also provides translational invariance, allowing the model to recognise features even if their position shifts slightly. Common methods like max pooling or average pooling summarize local regions to retain the most critical information.

③ Define stride and padding in CNN.

① Stride is the number of pixels the convolutional filter moves or "steps" across the input image at a time; a higher stride results in a smaller

output volume.

Stride controls the density of the feature map, effectively "skipping" data to reduce dimensionality.

④ padding involves adding extra (usually zero-valued) pixels around the border of the input image to control the spatial size of the output.

It prevents the shrinking of the feature map & ensures that information at the edges of the image is processed as thoroughly as the center.

Together, they determine the final dimensions of the feature map via the formula:

$$O = \left(\frac{W - K + 2P}{S} \right) + 1$$

④ What are activation functions importance in CNN?

Activation functions ~~introduce~~ introduce non-linearity into the network, allowing it to learn complex patterns rather than just simple linear transformations. Without them, no matter how many layers a CNN has, it would behave like a single-layer linear model, unable to map intricate images to features. They act as "gates" that determine whether a neuron should fire based on the input's relevance to specific feature being detected. Modern functions like ReLU also help mitigate the vanishing gradient problem, enabling deeper architectures and faster training. Effectively, they transform the linear output of convolutions into a sophisticated decision-making signal.

⑤ Explain the role of fully connected layers in CNN.
In convolutional neural network (CNN), fully connected (FC) layers act as the "classifier".

That step at the end is the pipeline. While earlier layers extract spatial features like edges and textures, the FC layer glues these high-level features into a single vector. It then applies weights to these features to determine their global importance, mapping them to the final output classes. Essentially, it looks at the "vocabulary" of features gathered by convolutions to make a final decision on what the image actually represents.

⑥ A CNN model is overfitting on image data.

Suggest techniques to improve generalization. To improve generalization and reduce overfitting in a CNN, apply ~~these~~ these techniques:

1. Data Augmentation: Artificially expand your dataset by applying random rotations, flips, zooms, and color shifts to training images.
2. Regularization: Implement Dropout layers to randomly deactivate neurons during training or use weight decay (L2) to penalize large weights.
3. Batch Normalization: Normalize the inputs of each layer to stabilize training, speed up convergence, and provide a slight regularization effect.
4. Early Stopping: Monitor validation loss during training and halt the process as soon as performance begins to degrade to prevent the model from memorizing noise.
5. Simplified Architecture: Reduce the model's capacity by removing layers or decreasing

The number of filters is network is too complex for size of your dataset.

⑦ You need to design a CNN for medical image classification with limited data. What strategies would you use?

To handle limited medical data, I would prioritize transfer learning by fine-tuning a pre-trained model like ResNet or EfficientNet on ImageNet weights. I'd implement aggressive data augmentation (rotations, flips, and elastic deformations) to artificially expand the training set. Additionally, using dropout and weight decay would prevent overfitting, while global average pooling would reduce the number of trainable parameters. Finally, I would use cross-validation to ensure the model generalizes well across the entire small dataset.

Chapter 4 NLP

① Explain TF-IDF and its significance.

TF-IDF (Term Frequency - Inverse Document Frequency) is a numerical statistic that measures how important a word is to a specific document within a larger collection (corpus). It multiplies Term Frequency (how often a word appears in a doc) by Inverse Document Frequency (how rare the word is across all docs). This effectively filters out common "stop words" like "the" or "is" that carry little information while boosting unique, descriptive keywords. Its significance lies in converting text into meaningful vectors for machine learning, enabling tasks like search engine ranking and document clustering.

② What are word embeddings and how do they differ from TF-IDF?

Word embeddings (like word2vec or GloVe) represent words as dense, multi-dimensional vectors in continuous space where geometric distance reflects semantic similarity. Unlike TF-IDF, which is a sparse, frequency-based statistical measure of a word's importance to a document, embeddings capture contextual meaning and relationships (e.g., "king" - "man" + "woman" = "queen"). While TF-IDF treats words as isolated tokens (ignoring synonyms), embeddings allow models to understand that "happens" and "gas sub" are related. Essentially, TF-IDF measures relevance within a corpus, whereas embeddings map the ~~abstract~~ conceptual essence of language.

③ What is tokenization and why is it important?

Tokenization is the process of breaking raw text into smaller units called tokens, such as words, characters, or subwords. It is critical because machines cannot process raw strings; they require these discrete chunks to convert text into numerical vectors. By defining the "vocabulary" of a model, tokenization enables the analysis of grammatical structures and semantic meaning. Without it, a model couldn't distinguish between individual words or understand where a thought begins and ends.

④ DISCULDS The role of stopwords in NLP preprocessing. Stopwords are high-frequency words (like "the", "is", "in") that carry little unique semantic meaning and are often removed to reduce data noise.

By Silencing them, you decrease the vocabulary size and computational complexity, allowing models to focus on the more informative "content" words. However, their removal can be detrimental in tasks like sentiment analysis or machine translation, where words like "not" or "but" are crucial for understanding context. Ultimately, the decision to remove them depends on whether the task requires structural grammar or just keyword-based themes.

⑤ Compare BoW vs word embeddings.

- Bag-of-words (BoW) creates sparse, high-dimensional vectors based on raw word counts, completely ignoring word order and ~~the~~ semantic meaning.
- Word embeddings (like word2vec) produce dense low-dimensional vectors that capture linguistic relationships and context by mapping similar words closer together.
- While BoW treats "run" and "ran" as entirely unrelated entities, embeddings recognize their shared meaning through vector proximity.
- BoW suffers from the "curse of dimensionality" as vocabulary grows, whereas embeddings maintain a fixed, efficient size regardless of dataset scale. Ultimately, BoW is better for simple ~~than~~ document classification, while embeddings are essential for complex tasks like translation or sentiment analysis.

⑥ You are building a sentiment analysis model. would you remove stopwords? justify your answer

Removing stopwords is not recommended for modern

Date.....
Page.....

sentiment analysis (like transformers or lstms) because words like "not," "no," or "but" are crucial for understanding context and negation, while it was once common to remove them to reduce "noise" and save compute in simpler models, doing so today often destroys the emotional meaning of a sentence.

For instance, removing "not" from "not happy" turns negative sentiment into a positive one. Only remove them if you are using basic frequency-based models (like naive Bayes) where computational efficiency is a higher priority than nuanced accuracy.

⑦ A chatbot fails to understand context despite good accuracy. What NLP improvements would you suggest?

To fix a chatbot that lacks "memory" despite high accuracy, I suggest these NLP improvements:

1. Implement Transformer Architecture & use models like BERT or GPT that utilize self-attention to weigh the importance of preceding words in a sequence.
2. Add Conversation History Buffers: pass the last k turns of dialogue back into the model to provide immediate context for the current query.
3. Utilize Coreference Resolution & Integrate rules to help the model identify what "it," "they," or "that" refers to from previous sentences.
4. Incorporate state management & use Hidden Markov models (HMM) or persistent session variables to track the user's intent across multiple interactions.
5. Fine-tune on domain-specific dialogue & train on long-form conversational datasets rather than

isolated question-answer pairs to improve "slow" text recognition.

Chapter 5 Recommendation Systems

① Explain Content-based Filtering.

Content-based filtering recommends items to user by matching their past preferences with specific item attributes (like genre, actor, or keywords). It simply builds a user profile based on the features of items the user liked before. It is highly effective for recommending niche items but can suffer from "filter bubbles" where the user is never exposed to new categories.

② Describe Collaborative Filtering.

Collaborative filtering is a recommendation technique that predicts a user's interest by collecting preferences from many other users. It operates on the principle that if a person A and person B agree on one issue, they are likely to agree on others. Crucially, it requires no knowledge of the items themselves (like genre, or color), relying on user-item interaction patterns.

③ What is the cold-start problem?

The cold-start problem occurs in recommendation systems when there is insufficient data to make accurate predictions. It affects new users (no history) and new items (no interactions), forcing the system to rely on metadata or popular trends until more data is gathered.

④ Explain matrix factorization in recommendation systems.

Matrix factorization decomposes a large user-item interaction matrix into two lower-dimensional matrices representing latent features (e.g., "genre" or "vibe") for users and items. By calculating the dot product of these two vectors, the model predicts missing values to estimate a

Date.....
Page.....

Users interact in on item they haven't seen. This technique, famously used in the Netflix prize, effectively uncovers hidden patterns in sparse data to deliver personalized recommendations.

⑤ What are hybrid recommendation systems?

Hybrid recommender systems combine two or more recommendation schemes, typically Collaborative Filtering and Content-Based Filtering - to leverage the strengths of each while neutralizing their individual weakness. They are specifically designed to overcome common issues like the cold-start problem.

⑥ A new e-commerce platform has no user data. How would you design a recommendation system?

I would implement a content-based system using product metadata (categories/descriptions) and knowledge-based recommendations via user preference quizzes to address the cold start problem. To maximize discovery, I'd use popularity-based filtering to show trending items and Association Rule mining ("frequently bought together") as soon as the first user transaction occurs. This avoids the need for historical user profiles while providing immediate, relevant value.

⑦ A new recommender system shows repetitive suggestions. How would you improve diversity to improve diversity and break "filter bubbles" if you implement maximal marginal relevance (mmr) to re-rank items by balancing relevance against similarity to already selected results. You can also introduce exploration-exploitation strategies, such as ϵ -greedy or Thompson Sampling, to inject novel or serendipitous content into the feed. Additionally,

incorporating collaborative-filtering with diversification penalties or using "long-tail" items ensures the system doesn't just give privileges toward the most popular, repetitive entries.

Chapter 6 Reinforcement Learning

① What is reinforcement learning and how is it different from supervised learning?

Reinforcement Learning (RL) is a goal-oriented approach where an agent learns to make decisions by performing actions in an environment to maximize a cumulative reward. Unlike supervised learning, which relies on a static dataset or labeled "right answers," RL learns through trial and error and feedback from its own experiences. While supervised learning maps ~~static~~ inputs to outputs based on historical examples, RL maps states or actions based on future potential gains.

② Define policy, reward, and value function?

- Policy (π): The strategy or mapping that tells the agent which action to take based on its current state.
- Reward (R): The immediate scalar feedback (positive or negative) the agent receives from ~~value~~ the environment after an action.
- Value function (V): The prediction of the total long-term accumulated reward an agent can expect to gain starting from a specific state.

③ Explain Q-learning with an example.

Q-learning is a model-free reinforcement learning algorithm where an agent learns the value of taking specific actions in specific states to maximize long-term rewards. It maintains a Q-table to store "quality" scores, updating them as the agent explores and

receives feedback from the environment. For example, a robot in a maze earns +10 for finding cheese (reward) and -10 for hitting a wall (penalty). Over time, the robot updates its Q-values to choose the path with the highest cumulative score, effectively "mapping" the best route through trial and error.

④ What is the exploration - exploitation tradeoff?

The exploration-exploitation tradeoff is the dilemma of choosing between a known good option and an unknown potentially better one. Exploitation involves choosing the best-performing action discovered so far to maximize immediate rewards. Exploration involves trying new or sub-optimal actions to gather more information, which might reveal higher long-term gains. Balancing them is crucial; too much of exploitation leads to stagnation, while too much of exploration prevents you from ever ~~crashing~~ settling in on your knowledge.

⑤ What is the role of discount factor in RL?

The discount factor (γ) determines how much the agent values future rewards compared to immediate ones. It typically ranges from 0 to 1, where a value near 0 makes the agent "short-sighted," prioritizing long-term success and cumulative gains. Mathematically, it prevents infinite reward sums in continuing tasks and ensures the model convergence towards an optimal strategy.

⑥ A robot is not learning optimal behaviour due to sparse rewards. How would you redesign the reward system?

To fix sparse rewards, implement reward shaping by

adding intermediate, "dense" rewards for progress milestones that guide the agent toward the goal. You could also use curriculum learning, starting the robot with simpler tasks and gradually increasing difficulty as it learns. hindsight Experience Replay (HER) is another powerful trick, where the agent learns from "successful" attempts by treating the robot to explore novel states even when environmental feedback is absent.

❗ In a game-playing agent, the model has always exploits known state strategies. How would you improve exploration?

To encourage exploration, you can implement an ϵ -greedy strategy, where the agent chooses a random action with probability ϵ , instead of the best known one. Alternatively, upper confidence Bound (UCB) selection can be used to prioritize actions with high uncertainty, ensuring every move is tried eventually. You could also apply entropy regularization in policy gradients to discourage the model from converging too quickly on a single strategy.

Finally, curiosity-driven learning rewards the agent for reaching "surprising" or novel states, effectively incentivizing the discovery of new game mechanism mechanics. These methods prevent the agent from getting stuck in local optima by forcing it to test the environment's boundaries.